

GTD-FocalResNet: A Residual Focal-Loss Neural Model for Attack Success Prediction

Prepared by: Abelardo, Aquino, Balingit, & Gumban. | Course & Section: 3CSD – Machine Learning | Date: May 2026

GitHub Repository: <https://github.com/Shrxmy/CSELEC2C-FinalProject>

ABSTRACT

Predicting geopolitical violence remains a highly complex challenge, necessitating evaluation of model performance across varying decision boundaries. This study compares five modeling architectures, a class-weighted Logistic Regression, CART, XGBoost, a standard feedforward neural network, and a proposed task-specific GTD-FocalResNet using the Global Terrorism Database (GTD). To prevent data leakage, variables describing post-attack consequences were strictly excluded, and all models were evaluated under a strict temporal train-validation-test split. Model performance was evaluated using ROC-AUC, F1 Score, Precision, and Recall, while threshold simulations were used to investigate error distributions. Under the default classification threshold, the standard feedforward neural network achieved the highest temporal F1 score at 0.898, closely followed by the proposed GTD-FocalResNet with an F1 score of 0.896, demonstrating that the custom residual focal-loss design serves as a highly competitive and viable task-specific adaptation for imbalanced and temporally shifted tabular data. XGBoost produced the strongest traditional machine-learning baseline with an F1 score of 0.877, while Logistic Regression and CART provided stable, interpretable baselines. Threshold analysis demonstrated that adjusting decision boundaries significantly alters classification trade-offs. These findings show that while nonlinear machine learning models can improve predictions, they capture patterns in recorded events rather than the underlying reality, highlighting the necessity of threshold awareness in geopolitical forecasting.

Keywords: Global Terrorism Database; attack-success prediction; GTD-FocalResNet; focal loss; residual neural network; temporal validation; class imbalance; XGBoost; permutation feature importance

I. INTRODUCTION

Machine learning is increasingly deployed to analyze large, complex, and rapidly expanding datasets. Geopolitical violence presents a particularly challenging domain for these applications. Predictive modeling in this context is not solely a technical problem; the data is derived from real events, media reports, institutional records, and specific coding decisions. Consequently, a model trained on these records does not learn the absolute reality of geopolitical violence. Rather, it learns the patterns of what was observed, reported, and successfully entered into a database (Drakos & Gofas, 2006).

This study utilizes the Global Terrorism Database (GTD) to provide a large-scale historical record of terrorist incidents. The primary analytical task is to predict whether a recorded attack is classified as "successful". In the GTD, a successful attack indicates that the immediate tactical goal was completed, but it does not necessarily measure long-term political impact, social harm, public fear, or broader strategic value. Therefore, treating the dataset requires an understanding of real-world complexity: a failed attack may still inflict casualties and generate fear, while a successful attack might

be highly localized and small in scale. Furthermore, reporting disparities exist across different regions, making some events easier to classify than others (Weidmann, 2016).

This real-world complexity directly impacts the interpretation of predictive errors. In standard classification tasks, false positives and false negatives are treated as simple metric counts; in geopolitical forecasting, these errors carry significant weight. A false positive may erroneously inflate the perceived risk of a specific context, whereas a false negative could obscure an escalating tactical pattern that security models should detect. While this study does not assign specific policy costs to these errors, the inherent trade-off informs the evaluation framework.

To manage this class imbalance and evaluate these trade-offs, this study systematically compares five modeling architectures. Logistic regression serves as the interpretable baseline, providing readable coefficients and transparent linear assumptions (Hosmer, Lemeshow, & Sturdivant, 2013). A Classification and Regression Tree (CART) is utilized to capture non-linear decision rules while maintaining high interpretability (Breiman et al., 1984). XGBoost is implemented because boosted tree ensembles demonstrate robust performance on tabular data and effectively capture complex feature interactions (Chen & Guestrin, 2016). A standard feedforward neural network provides a highly flexible, deep architecture baseline (Goodfellow, Bengio, & Courville, 2016). Finally, building upon these foundations, a proposed residual focal-loss architecture (GTD-FocalResNet) is introduced to address the specific challenges of tabular imbalance.

By treating the decision threshold as an active experimental variable rather than a static default, this project challenges the notion that security-related predictive modeling is a purely objective, technical exercise. Every modeling choice carries embedded assumptions that shape the final output, and this study aims to explicitly expose these choices.

II. RELATED WORK

The Global Terrorism Database (GTD) serves as an extensive open-source event repository widely utilized for large-scale modeling (LaFree & Dugan, 2007). However, terrorism measurement is inherently sensitive to how incidents are coded and interpreted across time (Enders, Sandler, & Gaibullov, 2011), and event counts are frequently distorted by underreporting bias where certain incidents are less likely to appear in the record (Drakos & Gofas, 2006). Consequently, variables such as attack success reflect operational database outcomes governed by specific institutional rules rather than a universal measure of strategic effectiveness (START, 2022).

In predictive modeling, architectures must balance complexity with interpretability. Logistic regression provides a standard, transparent baseline by modeling log-odds with inspectable coefficients (Hastie, Tibshirani, & Friedman, 2009), though care must be taken as uneven class distributions can distort probability estimates and threshold decisions (King & Zeng, 2001). Classification and

Regression Trees (CART) offer an interpretable non-linear comparison by recursively splitting the feature space to capture threshold effects (Breiman et al., 1984). For more complex feature interactions, XGBoost applies gradient boosting and regularization to build robust ensembles (Friedman, 2001; Chen & Guestrin, 2016), while standard feedforward neural networks provide highly flexible, layered representations (Goodfellow, Bengio, & Courville, 2016; Paszke et al., 2019). However, standard neural architectures often struggle with highly imbalanced tabular data, necessitating custom solutions like focal loss and residual dense blocks to properly weight difficult examples.

Evaluating these algorithms requires metrics beyond raw accuracy, which can be misleading under severe class imbalance. Precision-recall analysis often proves more informative than standard ROC analysis in such contexts (Davis & Goadrich, 2006; Saito & Rehmsmeier, 2015). Furthermore, because modern predictive models are frequently miscalibrated (Guo et al., 2017), threshold selection remains a critical component that controls the balance between false positives and false negatives. For opaque models like XGBoost and neural networks, tools like SHAP are necessary to assign feature contributions (Lundberg & Lee, 2017) and ensure explanations remain cognitively accessible to users (Ribeiro, Singh, & Guestrin, 2016; Miller, 2019). Finally, algorithmic fairness literature emphasizes that machine learning systems can produce disparate outcomes across subgroups even without explicit discriminatory intent (Barocas, Hardt, & Narayanan, 2019; Chouldechova & Roth, 2020). This necessitates rigorous subgroup evaluation to identify uneven error distributions across diverse geopolitical and tactical contexts (Suresh & Guttag, 2021).

III. MODELS & ALGORITHMS

The target variable in this study is the GTD success field, operationalized as a binary classification label. Input features were limited to information that could plausibly be known at or near the time of the recorded event, including timing, geography, attack type, target type, weapon type, suicide indicators, multiple-incident indicators, uncertainty fields, internationalization indicators, and claim-related fields. To prevent data leakage, post-event consequence variables such as casualty counts, property damage, ransom outcomes, and hostage outcomes were excluded.

This study compares models with different assumptions: a linear baseline, a single decision tree, a boosted-tree ensemble, a standard dense neural network, and a proposed residual focal-loss neural network. The GTD-FocalResNet is introduced as a task-specific methodological adaptation rather than as a new deep-learning theory. Its purpose is to test whether residual tabular learning, focal-loss reweighting, feature dropout, softened class weighting, and calibration can improve or stabilize neural prediction under class imbalance and temporal shift. The models are evaluated with threshold-based metrics such as accuracy, precision, recall, and F1 score, while ROC-AUC is retained as a threshold-free ranking metric.

3.1 Logistic Regression

Logistic regression serves as the transparent linear baseline. It estimates the log-odds of attack success as a weighted sum of the processed input features. Numeric variables are standardized, categorical variables are one-hot encoded, and class weighting is applied to reduce the effect of unequal class proportions. Its main advantage is interpretability: a positive coefficient increases the

estimated log-odds of success, while a negative coefficient decreases it, assuming the other encoded features remain fixed. The probability model is mathematically defined as:

Equation (1): Logistic regression probability model

$$p(y = 1 | x) = \sigma(z), \quad z = \beta_0 + \sum \beta_i x_i$$

$$\sigma(z) = 1 / (1 + e^{-z})$$

The linear predictor (z) calculates a score based on feature weights (β_i), where a positive coefficient indicates a feature that increases the probability of success. The sigmoid function (σ) maps this score to a probability between 0 and 1, facilitating its interpretation as a likelihood of success.

3.2 Classification and Regression Trees (CART)

CART is the single-tree nonlinear baseline. It recursively divides the feature space into regions with different success probabilities. The tree chooses splits that reduce node impurity, and the implementation limits depth and minimum leaf sizes to reduce overfitting. CART is useful because it is more flexible than logistic regression while remaining easier to inspect than a boosted ensemble.

Equation (2): CART Gini impurity and split gain

$$Gini(t) = 1 - \sum p(k | t)^2$$

$$Gain = Gini_{parent} - \left(\frac{n_{left}}{n} G_{left} + \frac{n_{right}}{n} G_{right} \right)$$

Gini Impurity measures the statistical "purity" of a node; a value of 0 indicates that all incidents in that node belong to a single class (e.g., all successful). Gain calculates the reduction in impurity achieved by a specific split. The model selects the feature and threshold that provide the maximum reduction in impurity to create the explicit decision boundary.

3.3 eXtreme Gradient Boosting (XGBoost)

XGBoost estimates the probability of success using an ensemble of gradient-boosted decision trees. Each tree learns patterns missed by previous trees, allowing the model to capture nonlinear interactions between tactical, target, weapon, and geographic features. Regularization and controlled tree depth help prevent the boosted ensemble from simply memorizing the training data.

Equation (3): XGBoost additive tree model

$$\hat{y}_i = \sigma(\sum f_j(x_i))$$

$$Loss = \sum_i loss(y_i, \hat{y}_i) + \sum \Omega(f_j)$$

In this formula, each f_j is one tree in the ensemble. The loss term rewards correct predictions, while $\Omega(f_j)$ penalizes overly complex trees. This lets XGBoost learn flexible decision boundaries while still controlling model complexity.

3.4 Feedforward Neural Network

The feedforward neural network is a multilayer perceptron trained on the same processed feature matrix. Hidden layers transform the input features into learned nonlinear representations, and the final sigmoid layer outputs a probability of attack success. It is included as a standard neural baseline for comparison with the proposed residual model.

Equation (4): Feedforward neural-network transformation

$$h^1 = ReLU(W^1 h^{l-1} + b^l)$$

$$\hat{y} = \sigma(W^L h^{L-1} + b^L)$$

The hidden-layer equation shows how each layer transforms the representation from the previous layer. The final sigmoid output converts the last hidden representation into a probability.

3.5 GTD-FocalResNet

The proposed GTD-FocalResNet adapts existing deep-learning components to the GTD setting. It extends the standard neural network with residual dense blocks, focal loss, feature dropout, softened class weighting, and temperature scaling. Residual connections help the network learn corrections to earlier representations, while focal loss emphasizes harder examples that may be underrepresented in an imbalanced dataset. The contribution is therefore methodological and application-specific: it tests a deep-learning design tailored to temporally shifted tabular terrorism-event data.

Equation (5): *GTD-FocalResNet focal-loss weighting*

$$FL = -a_i(1 - p_i)^{\gamma} \log(p_i)$$

Focal loss reduces the influence of easy examples and increases the relative weight of harder cases. This is useful when successful and failed attacks are not equally represented. The residual blocks, dropout, class-weight softening, and calibration steps complement this loss function by improving regularization and probability behavior without changing the GTD target definition.

3.6 Evaluation Metrics

The main reported threshold-based metrics are accuracy, precision, recall, and F1 score. ROC-AUC is also reported to measure ranking ability independently of the 0.50 classification threshold. Precision and recall are especially important because they show the trade-off between false alarms and missed successful attacks.

Equation (6): *Main threshold-based metrics*

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

$$Precision = \frac{(TP + TN)}{(TP + FP)}$$

$$Recall = \frac{TP}{(TP + TN)}$$

$$F1\ Score = 2 \cdot \frac{(Precision \times Recall)}{(Precision + Recall)}$$

These mathematical expressions demonstrate that evaluating a model strictly through overall accuracy obscures class-specific error distributions. Accuracy measures the total proportion of correct predictions, but precision and recall provide more detail. Precision isolates false positives, while recall isolates missed positive instances. The F1 score balances precision and recall. The ROC-AUC evaluation measures the ranking capacity of the classifiers independently of the default threshold setting, an important aspect of model evaluation.

IV. IMPLEMENTATION

The implementation is organized around a main notebook that contains the full workflow from data loading to model evaluation. The notebook is designed to be self-contained so that preprocessing, model definitions, training logic, and evaluation steps can be reviewed in one place. This structure supports reproducibility while keeping the research workflow transparent.

The workflow begins by verifying or downloading the required GTD data files. Because the raw files are large, the notebook separates dataset retrieval from cleaning. After the first successful run, the cleaned, selected dataset is cached so that subsequent runs can skip slow raw-file parsing and proceed directly to splitting and modeling.

Cleaning step combines the required GTD sources, removes duplicate event identifiers, and keeps the binary success label. It also converts numeric fields and fills in missing categorical values with an "Unknown" category. To reduce leakage, it removes post-event consequence variables. The resulting feature set is then used consistently across all models.

Main evaluation uses a temporal train-validation-test split. Earlier events are used for training, a middle period for validation, and later events for testing. The random split option is kept only for comparison, as it mixes historical periods and usually produces easier test conditions.

Preprocessing includes numeric imputation and scaling for continuous variables and categorical encoding for text-based GTD fields. Logistic regression, CART, XGBoost, the feedforward neural network, and GTD-FocalResNet are trained on the same leakage-controlled feature set, so differences in performance stem from model assumptions rather than from different input data.

The notebook includes both a lighter reproducible profile and a heavier profile (this is via the `TRAINING_PROFILE` setting) for longer final runs. Trained models get saved after they're fitted. Traditional machine-learning models are turned into standard model files, but PyTorch neural networks are saved as checkpoint files, along with their fitted preprocessors. This avoids keeping the neural models only in memory and makes later reloading more reproducible. The final outputs are limited to materials that directly support the research claims: score tables, confusion matrices, neural learning curves, and permutation feature-importance summaries. This keeps the implementation focused on the evidence needed for model comparison and methodological interpretation.

V. RESULTS

This section presents the outputs that are most relevant to the research claims, including temporal test-set scores, the comparison between temporal and random, confusion matrices, neural learning curves, and permutation feature importance. The sequence moves from overall performance to error trade-offs, generalization behavior, and interpretation, so the results are not treated as disconnected figures.

The default threshold was 0.50 and the temporal evaluation framework showed that the two neural architectures had the best performance metrics. The Feedforward Neural Network achieved a test F1 score of 0.898 and ROC-AUC of 0.805. GTD-FocalResNet achieved an F1 score of 0.896, showing that the custom residual focal-loss design is competitive with the standard neural baseline. XGBoost was still the best performing traditional machine-learning model, with better precision but lower recall than the neural models.

Table 1. Temporal test-set metrics at the default 0.50 threshold. Values are rounded to three decimals.

Model	ROC-AUC	Avg. Precision	Accuracy	Precision	Recall	F1	MCC
Logistic Regression	0.770	0.944	0.748	0.917	0.773	0.839	0.306
CART	0.741	0.930	0.742	0.904	0.778	0.836	0.259
XGBoost	0.809	0.958	0.800	0.917	0.841	0.877	0.381
Feedforward NN	0.805	0.951	0.829	0.909	0.888	0.898	0.365
GTD-FocalResNet	0.797	0.950	0.827	0.907	0.886	0.896	0.359

The latest temporal results, as shown in Table 1, favor the neural models, with the Feedforward Neural Network slightly outperforming and GTD-FocalResNet closely following. This means the model proposed should not be presented as a clear winner of the leaderboard. Its contribution is better characterized as a competitive, task-specific deep learning adaptation that tackles imbalance and temporal shift with focal loss, residual connections, dropout and calibration.

Temporal and Random Split Comparison

The random split comparison is retained as a robustness check rather than the main evaluation. Because a random split mixes years across training and testing, it gives models access to patterns that are historically closer to the test cases. The temporal split is therefore the stricter setting for forward-looking evaluation

Table 2. Temporal vs. random split test comparison. Delta values are the random split minus the temporal split.

Model	Temp ROC	Rand ROC	Delta ROC	Temp F1	Rand F1	Temp BalAcc	Rand BalAcc
Logistic Regression	0.770	0.825	0.055	0.839	0.855	0.692	0.747
CART	0.741	0.825	0.084	0.836	0.875	0.660	0.742
XGBoost	0.809	0.875	0.066	0.877	0.885	0.709	0.787
Feedforward NN	0.805	0.866	0.062	0.898	0.927	0.697	0.753
GTD-FocalResNet	0.797	0.864	0.066	0.896	0.928	0.692	0.749

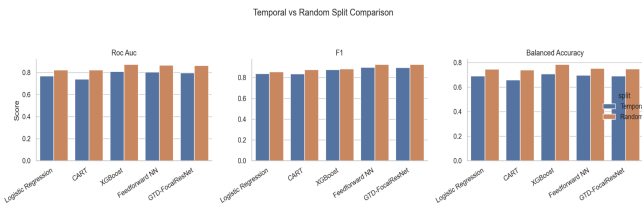


Figure 1. Temporal and random split comparison for ROC-AUC, F1, and balanced accuracy.

From Figure 1 and Table 2, we can see that random splitting improves the performance of models including GTD-FocalResNet. This proves that the random split makes the prediction task easier

and hence does not substitute temporal assessment. The temporal split remains the primary outcome as it more accurately reflects the challenge of learning on older records and testing on events at later times.

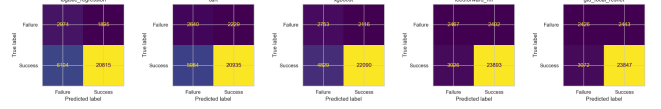


Figure 2: Confusion matrices for all five models on the temporal test set.

Trade-off in classification behind score table is shown in Figure 2. The neural models can discover more successful attacks, thus their better recall and F1 scores. XGBoost is more conservative and has strong precision, but it fails more successful cases than the neural models.

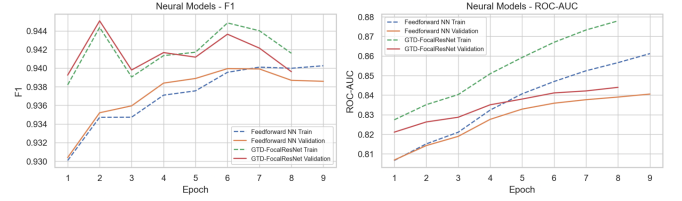


Figure 3. Neural learning curves for F1 and ROC-AUC under the temporal split.

In Figure 3 we see that tighter early stopping and regularization control the train-validation gaps of the neural models. The Feedforward Neural Network and GTD-FocalResNet keep strong validation F1 scores but the remaining ROC-AUC gap suggests that temporal generalization remains an open problem. This motivates validation-based checkpointing instead of simply training for more epochs.

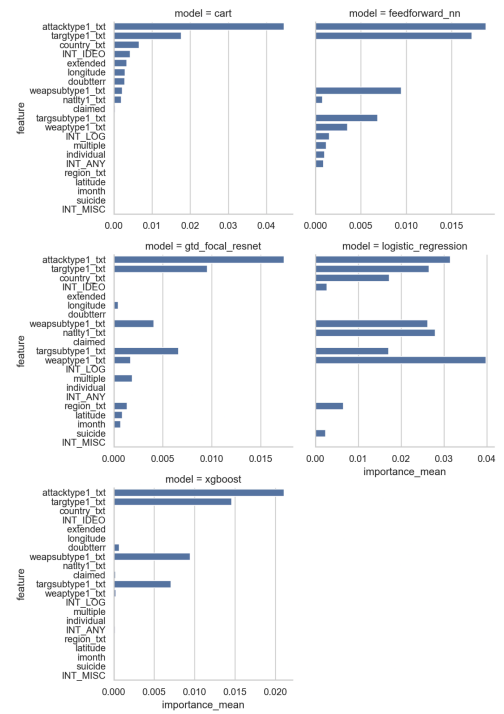


Figure 4. Permutation feature importance for all models.

The most useful predictors are mostly tactical and target-related variables, such as attack type, target type, weapon subtype and

target subtype (Figure 4). It is important to understand that these importance scores reflect predictive associations, and not causal effects. They also support the leakage-control strategy, as the strongest signals are from event descriptors, not post-event consequence variables.

VI. DISCUSSION

The main result is that neural models perform best under the temporal split by F1, while XGBoost remains the strongest traditional machine-learning baseline. In the latest run, the Feedforward Neural Network slightly outperforms GTD-FocalResNet, so the proposed model should be interpreted as a competitive methodological adaptation rather than a guaranteed top-performing architecture.

GTD-FocalResNet's strength lies in its design, combining residual dense blocks, focal loss, softened class weighting, feature dropout, and calibration to adapt established deep-learning components to an imbalanced, temporally shifted tabular prediction problem. This satisfies the study's methodological goal, even when the standard feedforward neural network is marginally ahead in a single run.

Temporal-versus-random comparison reinforces the importance of evaluation design. Random splitting gives better scores because it can put similar historical periods in both training and test sets, but that's not what you want if you're trying to make claims about the future. The temporal split is difficult to predict, but it's more suitable for forward-looking research. Neural learning curves show that overfitting gets controlled once you regularize stricter and stop early. Small F1 gaps mean stable thresholded performance, while larger ROC-AUC gaps remind us that ranking future events remains difficult. These curves justify validation-based checkpoint selection and caution against overtraining, making a good case for careful evaluation design.

The models mainly rely on fields like attack, target, and weapon. These make sense as predictors, but they're not direct causes of success. GTD records have reporting, coding, and historical context issues, so model outputs should support analysis, not replace human judgment.

Several limitations remain. That success label is simplified, some fields are uncertain or missing, and the models don't use full event narratives or external indicators. Future work could add more contextual features, stronger calibration checks, and subgroup analysis once the workflow is stable.

VII. CONCLUSION

This study evaluated machine-learning models for predicting the GTD-coded success outcome of recorded terrorist incidents. Under the temporal split, the neural models achieved the strongest overall results, with the Feedforward Neural Network maintaining a marginal lead over the competitive GTD-FocalResNet architecture. XGBoost remained the most robust non-neural benchmark, while Logistic Regression and CART established baseline reference points.

The project's primary contributions comprise the comparative performance ranking of the models, the task-specific architectural adaptation, and the temporal validation workflow. GTD-FocalResNet applies residual tabular learning, focal loss, softened class weighting, feature dropout, and calibration to a temporally shifted and imbalanced GTD prediction task. Confusion

matrices, learning curves, and permutation feature importance then help explain how the models behave beyond a single score.

The findings should still be interpreted cautiously. GTD success is an operational database label, not a complete measure of political or social impact, and the records may reflect reporting conditions, coding uncertainty, and historical context. Future work could improve the study by adding event narratives, external political or country-level indicators, a more robust calibration analysis, and broader subgroup analysis. Overall, the project demonstrates that carefully validated machine-learning models can reveal useful patterns in GTD records, but their outputs should support analysis rather than replace human judgment.

REFERENCES

- Barocas, S., Hardt, M., & Narayanan, A. (2019). Fairness and machine learning: Limitations and opportunities. fairmlbook.org. <https://fairmlbook.org/>
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). Classification and regression trees. Wadsworth International Group.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>
- Chouldechova, A., & Roth, A. (2020). A snapshot of the frontiers of fairness in machine learning. Communications of the ACM, 63(5), 82-89. <https://doi.org/10.1145/3376898>
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In Proceedings of the 23rd International Conference on Machine Learning (pp. 233-240). Association for Computing Machinery. <https://doi.org/10.1145/1143844.1143874>
- Drakos, K., & Gofas, A. (2006). In search of the average transnational terrorist attack venue. Defence and Peace Economics, 17(2), 73-93. <https://doi.org/10.1080/10242690500445387>
- Drakos, K., & Gofas, A. (2006). The devil you know but are afraid to face: Underreporting bias and its distorting effects on the study of terrorism. Journal of Conflict Resolution, 50(5), 714-735. <https://doi.org/10.1177/0022002706291051>
- Enders, W., Sandler, T., & Gaibullov, K. (2011). Domestic versus transnational terrorism: Data, decomposition, and dynamics. Journal of Peace Research, 48(3), 319-337. <https://doi.org/10.1177/0022343311398926>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189-1232. <https://doi.org/10.1214/aos/1013203451>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press. <https://www.deeplearningbook.org/>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (pp. 1321-1330). PMLR. <https://proceedings.mlr.press/v70/guo17a.html>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
- Hosmer, D. W., Jr., Lemeshow, S., & Sturdivant, R. X. (2013). Applied logistic regression (3rd ed.). John Wiley & Sons. <https://doi.org/10.1002/9781118548387>
- King, G., & Zeng, L. (2001). Logistic regression in rare events data. Political Analysis, 9(2), 137-163. <https://doi.org/10.1093/oxfordjournals.pan.a004868>
- LaFree, G., & Dugan, L. (2007). Introducing the Global Terrorism Database. Terrorism and Political Violence, 19(2), 181-204. <https://doi.org/10.1080/09546550701246817>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In I. Guyon et al. (Eds.), Advances in Neural Information Processing Systems 30. <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
- Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. Artificial Intelligence, 267, 1-38. <https://doi.org/10.1016/j.artint.2018.07.007>
- National Consortium for the Study of Terrorism and Responses to Terrorism (START). (2022). Global Terrorism Database codebook: Methodology, inclusion criteria, and variables. University of Maryland. <https://www.start.umd.edu/gtd/>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems 32. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1135-1144). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939778>
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), Article e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Suresh, H., & Gutttag, J. V. (2021). A framework for understanding sources of harm throughout the machine learning life cycle. In Equity and Access in Algorithms, Mechanisms, and Optimization. Association for Computing Machinery. <https://doi.org/10.1145/3465416.3483305>
- Weidmann, N. B. (2016). A closer look at reporting bias in conflict event data. American Journal of Political Science, 60(1), 206-218. <https://doi.org/10.1111/ajps.12207>

APPENDICES

Appendix A. Member Contributions

Table A1 summarizes the main contributions of each group member. All members reviewed the final manuscript and notebook outputs before submission.

Member	Primary Responsibilities	Specific Outputs / Contributions
Abelardo, A. S.	Research framing, manuscript organization, and interpretation of terrorism-event context.	Helped define the study motivation, reviewed Introduction/Related Work framing, and checked that claims about GTD attack success were cautious and not causal overclaims.
Aquino, M. B.	Main provider of the topic for the project. Data preparation, split design, and reproducibility checks.	Provided the context of the whole project. Supported GTD cleaning decisions, temporal/random split processing, processed split-file organization, and verification of dataset cache and train/validation/test outputs.
Balingit, D. M.	Documentation, result presentation, and final quality review.	Helped polish the manuscript, align tables/figures with the latest results, prepare references/captions, and review the final notebook/document for submission readiness.
Gumban, J. P.	Model implementation, neural-network experimentation, and result generation.	Worked on Logistic Regression, CART, XGBoost, Feedforward NN, and GTD-FocalResNet runs; generated model metrics, confusion matrices, learning curves, and feature-importance outputs.

Appendix B. Reproducibility and Output Checklist

The final implementation notebook is self-contained and can be reproduced as the main temporal evaluation and the random-split comparison.

The main paper setting uses `SPLIT_MODE = "temporal"`, `CACHE_DATA = True`, `SAVE_PROCESSED_SPLITS = True`, and `REBUILD_DATA_CACHE = False`.

Item	Location / Description
Main notebook	notebooks/3CSD_Group 8_Implementation.ipynb
Cleaned data cache	data/processed/gtd_light_full.pkl
Processed temporal splits	data/processed/temporal/
Processed random splits	data/processed/random/
Temporal scores	results/light_reproducible/temporal/tables/scores.csv
Confusion matrices	results/light_reproducible/temporal/figures/confusion_matrices.png
Neural learning curves	results/light_reproducible/temporal/figures/learning_curve_neural_models_fl_roc_auc.png
Permutation importance	results/light_reproducible/temporal/figures/feature_importance_all_models.png
Temporal vs. random comparison	results/figures/temporal_vs_random_split_comparison.png

Appendix C. Final Model Set

The final comparison includes five models: Logistic Regression, CART, XGBoost, Feedforward Neural Network, and GTD-FocalResNet. The temporal split is treated as the primary evaluation because it better reflects forward-looking prediction, while the random split is retained only as a comparison showing how mixed-year evaluation can produce easier results.